CSE-3079 EMBEDDED IoT SYSTEMS

WEEK 04

HARDWARE TIMERS & TIMER-BASED BUTTON DEBOUNCING

Dr. Nasir Mahmood | Fall 2026 | CS 5th A, B

INTERRUPTS DETECT EVENTS. TIMERS MEASURE TIME.

Week 3 (The Flaw)



GPIO event → Interrupt → ISR → Immediate response

⚠ **Problem:** The ESP32 sees electrical edges, not human intention, leading to mechanical bounce misfires.

Week 4 (The Solution)



Precise Elapsed-Time Mechanism

GPIO ISR → Start timer → Wait without blocking → Timer ISR → Validate signal

Core Idea: A GPIO interrupt tells the ESP32 that something happened.
A timer tells the ESP32 when enough time has passed.

WEEK 3 RECAP: THE RULES OF ISR EXECUTION



SHORT & DETERMINISTIC

ISRs must execute instantly. No lengthy calculations.



NO BLOCKING

Never use delay() inside an ISR. It halts the CPU.



THE VOLATILE KEYWORD

Variables shared between the ISR and normal program flow must be marked volatile to prevent compiler caching.



IRAM_ATTR

ISR functions should be loaded into instruction RAM for fast execution.

THE FATAL FLAW OF DELAY()

Blocking delay():



Stops the current execution flow. Terrible fit for an ISR. Makes IoT applications unresponsive.

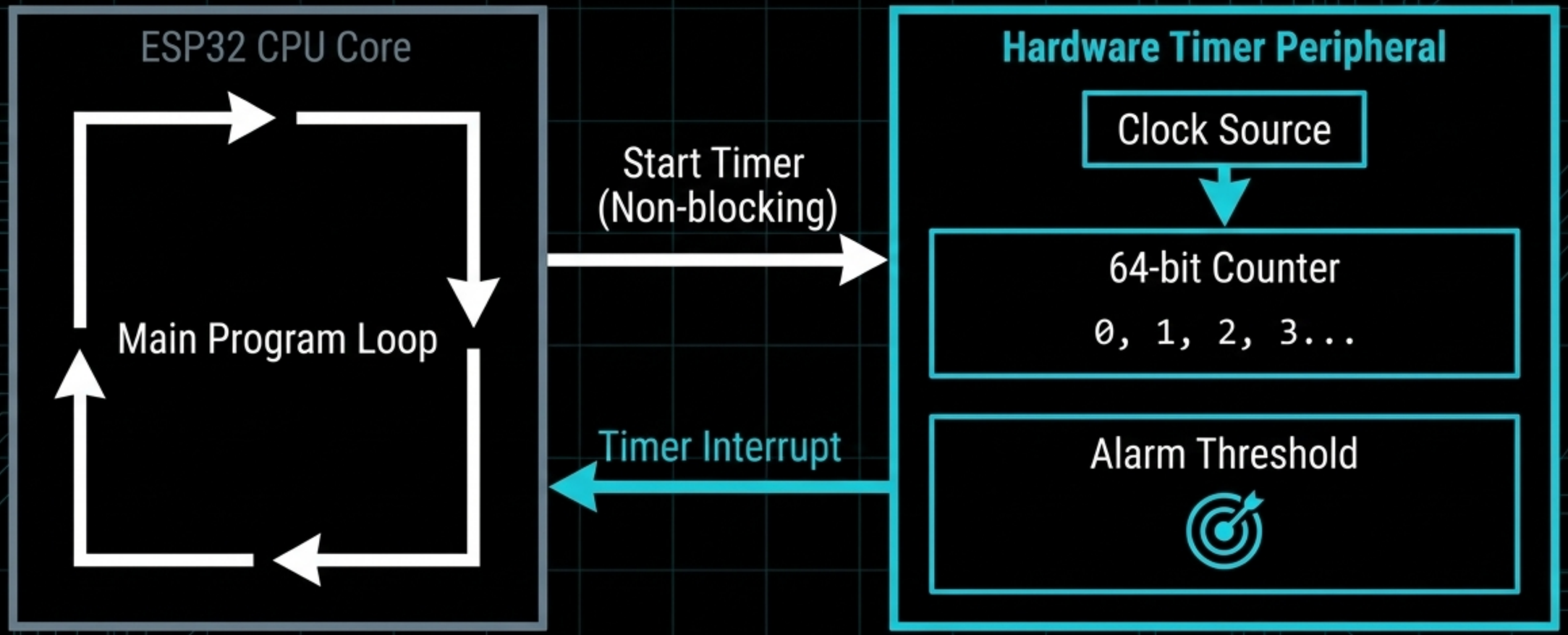
Hardware Timer:



Counts time independently. Generates an interrupt only when the interval expires.

Embedded Principle: Do not confuse 'waiting for time to pass' with 'making the CPU do nothing.'

HARDWARE TIMER FUNDAMENTALS



- A hardware timer is a dedicated silicon peripheral.
- It counts clock ticks independently of the normal application flow.
- When the counter hits the configured limit, it fires a Timer Event / ISR.

THE ANATOMY OF TIME: FREQUENCY AND TICKS

Base Setup

80 MHz clock with a 1 MHz prescaler = 1,000,000 ticks per second.

Tick Duration

$$T_{tick} = \frac{1}{f} \rightarrow \frac{1}{1{,}000{,}000}$$

= 1 microsecond (**1us**)

Alarm Calculation

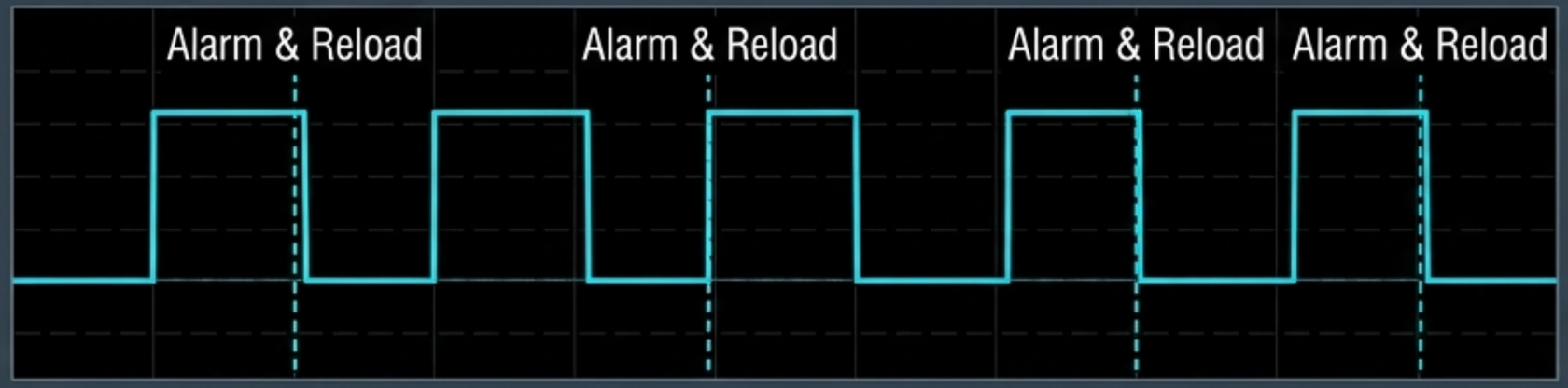
$$N = T_{event} \times f$$



Quick Reference Table
1 ms = 1,000 ticks
10 ms = 10,000 ticks
50 ms = 50,000 ticks (Standard Debounce)
1 s = 1,000,000 ticks

PERIODIC VS. ONE-SHOT TIMERS

Periodic (Auto-Reload)



Automatically resets and continues after hitting the alarm.

Use case: Blinking an LED, sampling a sensor periodically.

One-Shot



Generates a single alarm event and halts.

Use case: Single validation event after a delay (Timer-Based Debounce).

ARDUINO-ESP32 3.X TIMER API SYNTAX

Create Timer

```
timerBegin(frequency)
```

Example: timerBegin(1000000) requests a 1 MHz frequency directly.

Attach Callback

```
timerAttachInterrupt(timer, callback)
```

Example: timerAttachInterrupt(timer, &onTimer) binds the timer to your ISR.

Configure Alarm

```
timerAlarm(timer, value, autoreload, reload_count)
```

Example: timerAlarm(timer, 50000, false, 0) sets a 50,000-tick, one-shot alarm.

API EVOLUTION: 2.X VS 3.X

Arduino-ESP32 2.x (Old)

```
timerBegin(0, 80, true)
```

```
timerAttachInterrupt(timer,
  &onTimer, true)
```

```
timerAlarmWrite(…) +
 timerAlarmEnable(…)
```

Arduino-ESP32 3.x (Current)

```
timerBegin(1000000)
```
(Requests desired frequency directly)

```
timerAttachInterrupt(timer,
  &onTimer)
```

```
timerAlarm(timer, 1000000,
  true, 0)
```
(Enable is included)

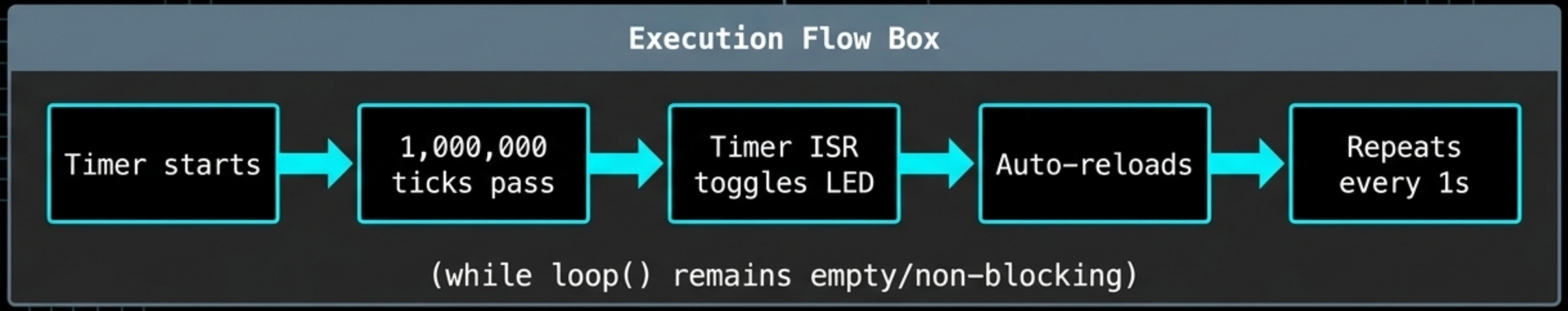
Version Note: Older examples use timer dividers. 3.x simplifies this. Always use the current API in this course.

PRACTICAL EXAMPLE: PERIODIC TIMER LED

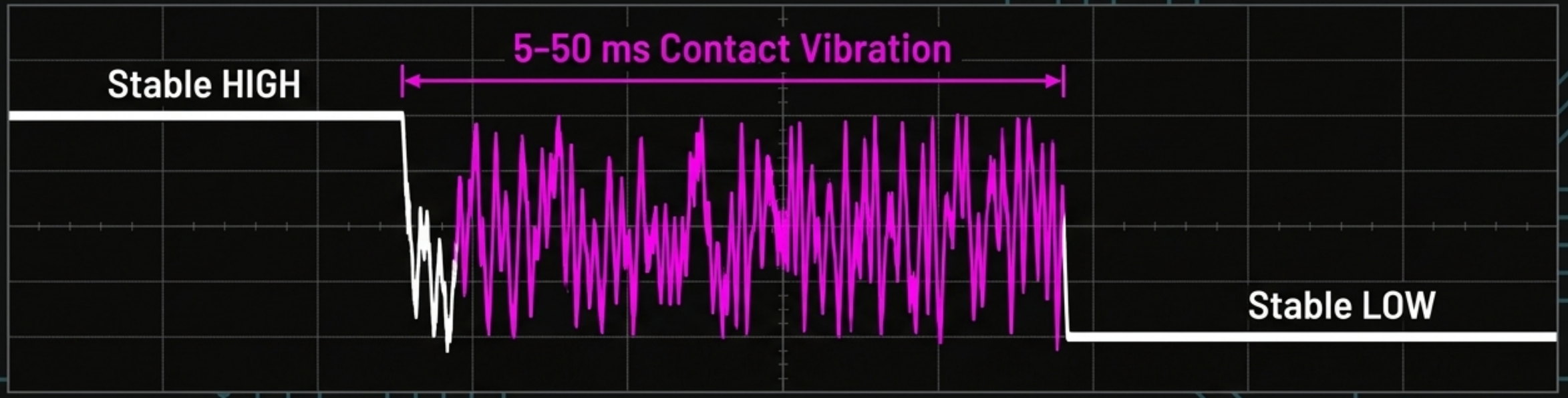
```
hw_timer_t *timer = NULL;

void ARDUINO_ISR_ATTR onTimer() {
  ledState = !ledState;
  digitalWrite(2, ledState);
}

timer = timerBegin(1000000);
timerAttachInterrupt(timer, &onTimer);
timerAlarm(timer, 1000000, true, 0);
```



THE PHYSICS OF MECHANICAL SWITCH BOUNCE



A mechanical button is not an ideal digital device.

Contacts vibrate when closing, causing the electrical signal to alternate rapidly between HIGH and LOW.

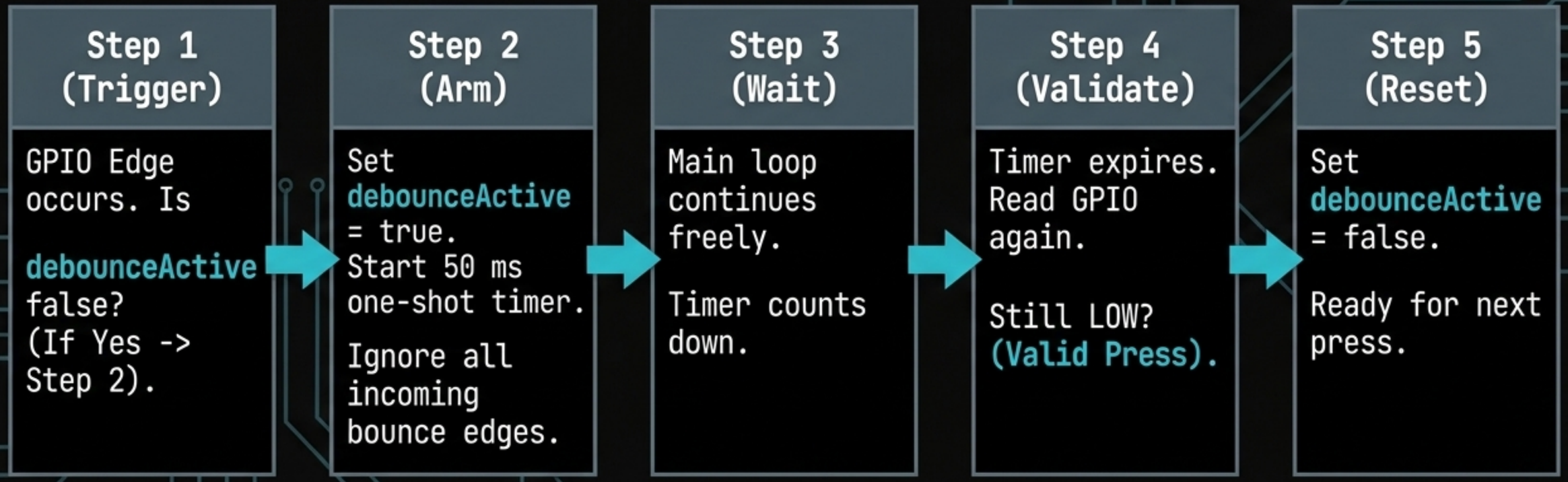
The Result: One physical human press generates multiple, unwanted GPIO interrupt edges.

The Goal: Decide which part of the observed signal represents human intention.

EVALUATING DEBOUNCING STRATEGIES

Hardware RC Filter	Software Delay (delay())	Timer-Based Non-Blocking
Smooths transition physically.	Waits post-edge.	Starts a one-shot timer and validates later.
Pro: Suppresses noise early.	**Pro:** Simple code.	**Pro:** Non-blocking, event-driven, perfect for interrupts.
Con: Requires extra board components.	**Con:** Blocking. Unsuitable inside an ISR.	**Con:** Requires careful ISR state design.

TIMER-BASED NON-BLOCKING DEBOUNCE LOGIC

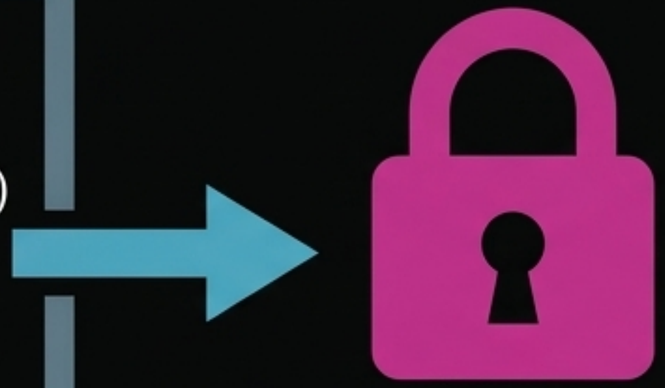


THE DUAL-ISR ARCHITECTURE

GPIO ISR
(Triggered by Physical Button)

Answers “Did an edge occur?”

Detects candidate press and starts the debounce interval.



```
volatile bool debounceActive
```

The Guard: This flag locks out all mechanical chatter while the timer is running.

Timer ISR
(Triggered by Timer Alarm)

Answers “Has the debounce interval elapsed?”

Validates the button state.

ESP32 DEBOUNCE IMPLEMENTATION: SETUP

```
hw_timer_t *debounceTimer = NULL;
volatile bool debounceActive = false;

void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  debounceTimer = timerBegin(1000000); // 1 MHz
  timerAttachInterrupt(debounceTimer, &onDebounceTimer);
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), onButtonISR, FALLING);
}
```

Note: We use INPUT_PULLUP, so a physical press connects to GND, generating a FALLING edge.

ESP32 DEBOUNCE IMPLEMENTATION: THE ISRs

GPIO ISR Block

```
void ARDUINO_ISR_ATTR onButtonISR() {
  if (!debounceActive) {
    debounceActive = true;
    timerWrite(debounceTimer, 0);
    timerAlarm(debounceTimer, 50000,
    false, 0); // 50ms One-Shot
  }
}
```

Timer ISR Block

```
void ARDUINO_ISR_ATTR onDebounceTimer() {
  if (digitalRead(BUTTON_PIN) == LOW) {
    // Valid press confirmed. Action goes
    here.
  }
  debounceActive = false;
}
```

LINE-BY-LINE LOGIC DECONSTRUCTION

```
volatile bool debounceActive = false;
```

volatile is mandatory. It prevents the compiler from caching the variable, ensuring the main program always reads the real-time value changed by the ISR.

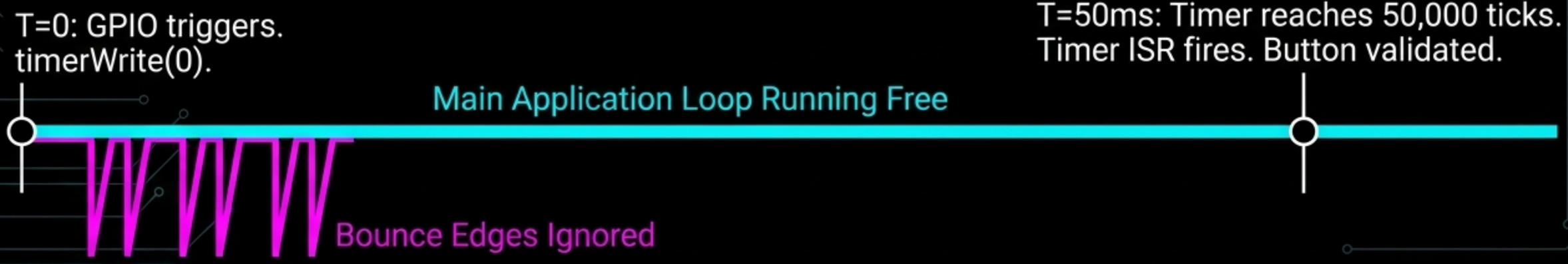
```
if (!debounceActive)
```

The **Gatekeeper**. Prevents multiple bounce edges from resetting the timer repeatedly.

```
digitalRead(...) == LOW
```

Hardware Physics. Because of **INPUT_PULLUP**, a released button reads **HIGH**. A pressed button reads **LOW**. We verify it is still LOW 50ms later.

TIMING CALCULATIONS & EXECUTION FLOW



The Math: 50 ms interval at 1 MHz base frequency

```
1 MHz = 1 us per tick.
50 ms = 50,000 us.
N = 50,000 ticks.
```

This non-blocking flow guarantees the CPU is never paused waiting for the mechanical vibration to stop.

LAB 1 SOLUTION: CODE

```
#define LED_PIN 2

hw_timer_t * timer = NULL;
volatile bool ledState = false;

void IRAM_ATTR onTimer() {
  ledState = !ledState;
  digitalWrite(LED_PIN, ledState);
}

void setup() {
  pinMode(LED_PIN, OUTPUT);
  // Configure timer: 1MHz frequency
  timer = timerBegin(1000000);
  // Attach interrupt handler
  timerAttachInterrupt(timer, &onTimer);
  // Set alarm: 1,000,000 ticks (1 second), autoreload = true
  timerAlarm(timer, 1000000, true, 0);
}

void loop() {
  // Main loop is free for other tasks
}
```

This code uses a hardware timer to toggle the LED every 1 second (1,000,000 ticks at 1MHz), leaving the main loop non-blocking.

COMMON ERRORS & DEBUGGING

SYMPTOM	CAUSE	FIX
timerBegin() compile error.	Using older 2.x API.	Ensure platform is updated; use timerBegin(frequency).
Timer repeats unexpectedly.	autoreload parameter set true.	Set to false for one-shot debounce timers.
Button triggers multiple times.	debounceActive guard missing.	Wrap timer start in if(!debounceActive).
Button logic inverted.	Misunderstanding INPUT_PULLUP.	Remember Released = HIGH, Pressed = LOW.

WEEK 4 TAKEAWAY & EXIT TICKET

The Master Concept: Interrupts detect events. Timers measure elapsed time. Combining them creates a precise, non-blocking Embedded IoT architecture.

Why not delay() in an ISR?
(It blocks execution, violating ISR rules).

What does 1 MHz mean for a timer?
(1 tick = 1 microsecond).

Why use a one-shot timer for debounce?
(You only want to validate the button once, not repeatedly).

Role of debounceActive?
(Prevents mechanical bounce from re-triggering the logic).